

Chapitre 2 : PL/SQL, Packages et Triggers

Procédures stockées et fonctions

Procédures stockées

Définition: procédures intégrées au SGBD

Exécutées dans une commande SQL

Ecrites dans un langage particulier, SQL, PL/SQL, C ...

Enregistrées dans le dictionnaire des données

Procédures stockées

Fonctions prédéfinies

- Fonctions numériques, mathématiques: AVG,COUNT, log, ln ...
- Fonctions sur caractères : initcap(), upper(), lower(), substr(), length(), trim, ||
- Fonctions de conversion: to_char, to_date ...
- Fonctions sur dates: SYSDATE ...

Fonctions diverses :

- Ex.

```
SELECT NomC, length(NomC) as Longueur      FROM Clients;
```

Deux types de fonction SQL

Les Fonctions Monolignes retournent un résultat par n-uplet

Les Fonctions Multilignes renvoient un résultat pour 1 ensemble de n-uplets

5

Fonctions monolignes

Fonctions de type caractère

Fonctions numériques

Fonctions de date

Fonctions de conversion

Fonctions générales

6

Fonctions de caractères

Changement de la casse des chaînes de caractères

- LOWER(chaine)
- UPPER(chaine)
- INITCAP(chaine)

7

Exemple

Affichez le numéro d'employé, le nom et le numéro de département de l'employé Higgins :

```
SELECT employee_id, last_name, department_id
FROM employees
WHERE last_name = 'higgins';
```

```
SELECT employee_id, last_name, department_id
FROM employees
WHERE LOWER(last_name) = 'higgins';
```

8

Manipulation de caractères

CONCAT(chaine1,chaine2)
SUBSTR(chaine,depart,longueur)
LENGTH(chaine)
INSTR(chaine,caractere)
LPAD(expression, longueur, motif)
RPAD(expression, longueur, motif)
REPLACE(chaine, chainerecherchee, chaineremplacement)
TRIM(caractere FROM chaine)

9

Exemple

```
SELECT employee_id, CONCAT(first_name, last_name) NAME,  
       job_id, LENGTH (last_name),  
       INSTR(last_name, 'a') "Contains 'a'?"  
FROM   employees  
WHERE  SUBSTR(job_id, 4) = 'REP';
```

10

Fonctions numériques

ROUND : Arrondi à un nombre de décimales donné
TRUNC : Tronque à un nombre de décimales donné
MOD : Retourne le reste de la division

11

Exemple ROUND

```
SELECT ROUND(45.923,2),  ROUND(45.923,4),  ROUND(45.923,0),  ROUND(45.923,-1),  
       ROUND(45.923,-2),  ROUND(51.923,-1), ROUND(51.923,-2), ROUND(51.923,-3),  
       ROUND(251.923,-3),  ROUND(551.923,-3)  
FROM   DUAL;
```

ROUND(45.923,2)	ROUND(45.923,4)	ROUND(45.923,0)	ROUND(45.923,-1)	ROUND(45.923,-2)
45.92	45.923	46	50	0
ROUND(51.923,-1)	ROUND(51.923,-2)	ROUND(51.923,-3)	ROUND(251.923,-3)	ROUND(551.923,-3)
50	100	0	0	1000

12

Exemple TRUNC

```
SELECT TRUNC(45.923,2), TRUNC(45.923),  
       TRUNC(45.923,-1)  
FROM   DUAL;
```

→45.92 45 40

13

Les dates

Stocke des dates selon un format numérique interne : siècle, année, mois, jour, heures, minutes et secondes

Le format par défaut d'affichage de la date est DD-MON-RR

Les dates Oracle valides appartiennent à l'intervalle 1er janvier 4712 avant J-C - 31 décembre 9999 après J-C

14

Exemple

```
SELECT last_name, hire_date  
FROM   employees  
WHERE  hire_date < '01-FEB-88';
```

15

Format de date RR

(documentation

Oracle)

Année en Cours	Date indiquée	Format RR	Format YY
1995	27-OCT-95	1995	1995
1995	27-OCT-17	2017	1917
2001	27-OCT-17	2017	2017
2001	27-OCT-95	1995	2095

		Si l'année à deux chiffres indiquée est :	
Si l'année à deux chiffres indiquée est :	0-49	La date renvoyée appartient au siècle en cours	La date renvoyée appartient au siècle précédant celui en cours
	50-99	La date renvoyée appartient au siècle suivant celui en cours	La date renvoyée appartient au siècle en cours

16

Fonction SYSDATE & CURRENT_DATE

SYSDATE est une fonction qui renvoie la date et l'heure dans le S.E. du serveur de BD

- Exemple 1 : SELECT sysdate FROM dual; -- affichage date simple
- Exemple 2 : SELECT TO_CHAR(SYSDATE, 'DD/MM/YYYY HH24:MI:SS') FROM DUAL; -- affichage avec heure

CURRENT_DATE est une fonction qui renvoie la date du jour de la session

- Exemple 1 : SELECT CURRENT_DATE FROM dual; -- affichage date simple
- Exemple 2 : SELECT TO_CHAR(CURRENT_DATE, 'DD/MM/YYYY HH24:MI:SS') FROM DUAL; -- affichage avec heure

SYSDATE	TO_CHAR(SYSDATE,'DD/MM/YYYYHH24:MI:SS')	CURRENT_DATE	TO_CHAR(CURRENT_DATE,'DD/MM/YYYYHH24:MI:SS')
9/29/2025, 3:41:00 PM	29/09/2025 15:41:00	9/29/2025, 3:41:00	29/09/2025 15:41:00

17

Opérations arithmétiques

Addition ou soustraction de date

Addition des heures à une date en divisant le nombre d'heures par 24

18

Opérateurs arithmétiques et dates

Exemple Oracle

```
SELECT last_name, (SYSDATE-hire_date)/7 AS WEEKS
FROM employees
WHERE department_id = 90;
```

19

Manipulation de dates

MONTHS_BETWEEN : Nombre de mois entre deux dates

ADD_MONTHS : Ajout de mois à la date

NEXT_DAY : Jour suivant la date indiquée

LAST_DAY : Dernier jour du mois

ROUND : Arrondi de la date

TRUNC : Troncature de la date

20

Exemples Oracle

MONTHS_BETWEEN ('01-SEP-95','11-JAN-94')

→19.6774194

ADD_MONTHS ('31-JAN-96',1)

→'29-FEB-96'

NEXT_DAY ('01-SEP-95','FRIDAY')

→'08-SEP-95'

LAST_DAY ('01-FEB-95')

→'28-FEB-95'

21

ROUND et TRUNC et dates

Exemples Oracle : Hypothèse : SYSDATE = '25-JUL-03'

ROUND(SYSDATE,'MONTH')

→ 01-AUG-03

ROUND(SYSDATE,'YEAR')

→ 01-JAN-04

TRUNC(SYSDATE,'MONTH')

→ 01-JUL-03

TRUNC(SYSDATE,'YEAR')

→ 01-JAN-03

22

Fonctions de conversion

Permettent de convertir EXPLICITEMENT les données d'un type vers un autre (par opposition aux conversions IMPLICITES)

23

Conversion implicite

De VARCHAR2 ou CHAR → NUMBER

De VARCHAR2 ou CHAR → DATE

De NUMBER → VARCHAR2 ou CHAR

De DATE → VARCHAR2 ou CHAR

24

Format Français

Sous live SQL, on peut modifier la session pour choisir le français avec NLS_LANG :

- ALTER SESSION SET NLS_LANGUAGE = 'FRENCH'; -- langue
- ALTER SESSION SET NLS_TERRITORY = 'FRANCE'; -- conventions du territoire (date, ...)
- ALTER SESSION SET NLS_CHARACTERSET = 'UTF8'; -- format des caractères
- Cf. docs Oracle
 - pour les DBA : la vue V\$NLS_PARAMETERS
 - Doc Oracle globale : <https://www.oracle.com/fr/database/technologies/faq-nls-lang.html>

PARAMETER	VALUE
NLS_LANGUAGE	AMERICAN
NLS_TERRITORY	AMERICA
NLS_CURRENCY	\$
NLS_ISO_CURRENCY	AMERICA
NLS_NUMERIC_CHARACTERS	.,-
NLS_CALENDAR	GREGORIAN
NLS_DATE_FORMAT	DD-MON-RR
NLS_DATE_LANGUAGE	AMERICAN
NLS_CHARACTERSET	AL32UTF8
NLS_SORT	BINARY
NLS_TIME_FORMAT	HH.MI.SS.XF AM
NLS_TIMESTAMP_FORMAT	DD-MON-RR HH.MI.SS.XF AM
NLS_TIME_TZ_FORMAT	HH.MI.SS.XF AM TZR
NLS_TIMESTAMP_TZ_FORMAT	DD-MON-RR HH.MI.SS.XF AM TZR
NLS_DUAL_CURRENCY	\$
NLS_NCHAR_CHARACTERSET	AL16UTF16
NLS_COMP	BINARY
NLS_LENGTH_SEMANTICS	BYTE
NLS_NCHAR_CONV_EXCP	FALSE

25

Format Français

Directement dans les fonctions de conversion :

- NLS_DATE_LANGUAGE=FRENCH
- NLS_NUMERIC_CHARACTERS

26

Fonction TO_CHAR

Syntaxe : **TO_CHAR(date, 'format_model')**

Avec

- Date : date à convertir
- Format_model :
 - Indiqué entre apostrophes
 - Différence entre majuscules & minuscules
 - Comporte un élément **fm** permettant de supprimer les espaces de complément ou les zéros de début
 - On peut ajouter des formatages sur les caractères avec les constantes fournies par Oracle NLS...

Exemple :

```
SELECT employee_id, TO_CHAR(hire_date, 'MM/YY') Month_Hired
FROM   employees
WHERE  last_name = 'Higgins';
```

27

Éléments du modèle de format de date

Élément	Résultat
YYYY	Année complète en chiffres
YEAR	Année en toutes lettres (en anglais)
MM	Mois sur deux chiffres
MONTH	Mois en toutes lettres
MON	Abréviation à trois lettres du mois
DY	Abréviation à trois lettres du jour de la semaine
DAY	Jour de la semaine en toutes lettres
DD	Jour du mois sous forme numérique

28

Exemples d'Elements de format

Élément	Description
SCC ou CC	Siècle. Le serveur utilise le préfixe - pour les dates avant J-C.
Années dans les dates YYYY ou SYYYY	Année. Le serveur utilise le préfixe - pour les dates avant J-C.
YYY ou YY ou Y	Dernier(s) chiffre(s) de l'année (3,2 ou 1).
Y.YYY	Année avec une virgule à la position indiquée.
IYYY, IYY, IY, I	Chiffres de l'année (4, 3, 2 ou 1), selon la norme ISO.
SYEAR ou YEAR	Année en toutes lettres. Le serveur utilise le préfixe - pour les dates avant J-C.
BC ou AD	Respectivement avant et après J-C.
B.C. ou A.D.	Respectivement avant et après J-C.
Q	Trimestre de l'année.

Élément	Description
MM	Représentation à deux chiffres du mois.
MONTH	Nom du mois complété avec des espaces pour atteindre une longueur de 9 caractères.
MON	Abréviation à trois lettres du nom du mois.
RM	Mois en chiffres romains.
WW ou W	Semaine de l'année ou du mois.
DDD ou DD ou D	Jour de l'année, du mois ou de la semaine.
DAY	Nom du jour complété avec des espaces afin d'atteindre une longueur de 9 caractères.
DY	Abréviation à trois lettres du nom du jour.
J	Jour julien (nombre de jours depuis le 31 décembre 4713 avant J-C).
IW	Semaines de l'année, selon la norme ISO (1 à 53).

29

Éléments du modèle de format de date

Exemple 1 : Partie H de la date

'HH24:MI:SS AM' 15:45:32 PM

Exemple 2 : Utilisation avec des chaînes de caractères :

'DD "of" MONTH' 12 of OCTOBER

Exemple 3 : ddsp et ddsph : jour en toute lettre et position

'ddsp' twenty-three

'ddsph' twenty-third

30

Fonction TO_CHAR avec des dates

Exemple Oracle : affichage nom et date d'embauche des employés avec un format de type Jour Mois et Année

```
SELECT last_name,
       TO_CHAR(hire_date, 'fmDD Month YYYY')
       AS HIREDATE
FROM   HR.employees;
```

LAST_NAME	HIREDATE
King	17 June 2013
Yang	21 September 2015
Garcia	13 January 2011
...	...

Idem mais avec le mois en minuscule

```
SELECT last_name,
       TO_CHAR(hire_date, 'fmdd month YYYY')
       AS HIREDATE
FROM   HR.employees;
```

LAST_NAME	HIREDATE
King	17 june 2013
Yang	21 september 2015
Garcia	13 january 2011

31

Fonction TO_CHAR avec des dates

Idem mais avec le mois tout en majuscule

```
SELECT last_name,
       TO_CHAR(hire_date, 'fmDD MONTH YYYY')
       AS HIREDATE
FROM   HR.employees;
```

LAST_NAME	HIREDATE
King	17 JUNE 2013
Yang	21 SEPTEMBER 2015
Garcia	13 JANUARY 2011

Affichage nom et date d'embauche des employés avec un format de type Jour Mois et Année en français

```
SELECT last_name,
       TO_CHAR(hire_date, 'fmDD Month YYYY',
              'NLS_DATE_LANGUAGE=FRENCH')
       AS HIREDATE
FROM   HR.employees;
```

LAST_NAME	HIREDATE
King	17 Juin 2013
Yang	21 Septembre 2015
Garcia	13 Janvier 2011

32

Fonction TO_CHAR avec des dates

Idem avec cette fois la date sous forme 17^{ème} de juin et l'année et l'heure Heure : Minute :
Seconde et indicateur anglo-saxon (AM/PM)

```
SELECT last_name,  
       TO_CHAR(hire_date,  
               'fmDdspth "of" Month YYYY fmHH:MI:SS AM')  
       HIREDATE  
FROM   HR.employees;
```

LAST_NAME	HIREDATE
King	Seventeenth of June 2013 12:00:00 AM
Yang	Twenty-First of September 2015 12:00:00 AM
Garcia	Thirteenth of January 2011 12:00:00 AM

34

Fonction TO_CHAR avec des dates

Idem avec cette fois la date sous forme 17^{ème} de juin et l'année et l'heure Heure : Minute :
Seconde et indicateur anglo-saxon (AM/PM) en Français

```
SELECT last_name,  
       TO_CHAR(hire_date, 'fmDdspth "jour de " Month YYYY fmHH:MI:SS AM',  
               'NLS_DATE_LANGUAGE=FRENCH' )  
       HIREDATE  
FROM   HR.employees;
```

LAST_NAME	HIREDATE
King	Seventeenth jour de Juin 2013 12:00:00 AM
Yang	Twenty-First jour de Septembre 2015 12:00:00 AM
Garcia	Thirteenth jour de Janvier 2011 12:00:00 AM

→ PL/SQL

35

Exemple

```
CREATE OR REPLACE FUNCTION jour_ordinal_fr(n NUMBER) RETURN VARCHAR2 IS  
BEGIN  
    CASE n  
        WHEN 1 THEN RETURN 'premier';  
        WHEN 2 THEN RETURN 'deuxième';  
        WHEN 3 THEN RETURN 'troisième'; ...  
        WHEN 31 THEN RETURN 'trente et unième';  
        ELSE RETURN TO_CHAR(n); -- Valeur par défaut si hors plage  
    END CASE;  
END;  
END;
```

36

TO_CHAR avec des nombres

Syntaxe : **TO_CHAR**(*number*, '*format_model*')
--

Élément	Résultat
9	Représente un nombre
0	Force l'affichage d'un zéro
\$	Place un signe dollar flottant
L	Utilise le symbole flottant de la devise locale
.	Affiche un point comme séparateur décimal
,	Affiche une virgule comme séparateur des milliers

37

Fonction TO_CHAR

Exemple

```
SELECT TO_CHAR(salary, '$99,999.00') SALARY
FROM   employees
WHERE  last_name = 'Ernst';
```

38

TO_NUMBER et TO_DATE

TO_NUMBER : Chaîne de caractères → nombre

```
TO_NUMBER(char[, 'format_model'])
```

TO_DATE : Chaîne de caractères → date

```
TO_DATE(char[, 'format_model'])
```

39

TO_CHAR et TO_DATE avec RR

Exemple Oracle : Trouver les employés embauchés avant 1990, utilisation du format de date RR, qui produit les mêmes résultats que la commande soit exécutée en 1999 ou maintenant :

```
SELECT last_name, TO_CHAR(hire_date, 'DD-Mon-YYYY')
FROM   employees
WHERE  hire_date < TO_DATE('01-Jan-90', 'DD-Mon-RR');
```

40

Exemple Oracle 1

```
SELECT last_name,
       UPPER(CONCAT(SUBSTR (LAST_NAME, 1, 8), '_US'))
FROM   employees
WHERE  department_id = 60;
```

41

Exemple Oracle 2

```
SELECT TO_CHAR(ROUND((salary/7), 2), '99G999D99',  
            'NLS_NUMERIC_CHARACTERS = ''.,''')  
      "Formatted Salary"  
FROM employees;
```

42

Fonction NVL

Convertit une valeur NULL en valeur réelle :

- Syntaxe : NVL (expr1, expr2)
- Expr1: source
- Expr2: cible
- Exemples :
 - NVL(commission_pct,0)
 - NVL(hire_date,'01-JAN-97')
 - NVL(job_id,'No Job Yet')

43

Fonction NVL2

Convertit une valeur NULL en valeur réelle

Syntaxe : NVL2 (expr1, expr2, expr3)

- *expr1* : valeur source qui peut contenir une valeur NULL.
- *expr2* : valeur renvoyée si *expr1* <> NULL
- *expr3* : valeur renvoyée si *expr1* = NULL.

44

Fonction NULLIF

Compare deux expressions et retourne NULL si elle sont égales sinon renvoie la première valeur

Syntaxe : NULLIF (expr1, expr2)

- *expr1* : doit avoir une valeur non NULL

45

Fonction COALESCE

Retourne la première expression non NULL de la liste

Syntaxe : COALESCE (*expr1, expr2, ... exprn*)

47

Exemples Oracle

```
SELECT last_name, salary, NVL(commission_pct, 0),  
       (salary*12) + (salary*12*NVL(commission_pct, 0)) AN_SAL  
FROM employees;
```

```
SELECT last_name, salary, commission_pct,  
       NVL2(commission_pct,  
            'SAL+COMM', 'SAL') income  
FROM   employees WHERE department_id IN (50, 80);
```

```
SELECT first_name, LENGTH(first_name) "expr1",  
       last_name, LENGTH(last_name) "expr2",  
       NULLIF(LENGTH(first_name), LENGTH(last_name)) result  
FROM   employees;
```

```
SELECT last_name, employee_id,  
       COALESCE(TO_CHAR(commission_pct), TO_CHAR(manager_id),  
                'No commission and no manager')  
FROM   employees;
```

48

Expressions conditionnelles

Logique IF-THEN-ELSE en SQL.

Deux fonctions disponibles

- CASE
- DECODE

49

CASE

Syntaxe

```
CASE expr WHEN comparison_expr1 THEN return_expr1  
         [WHEN comparison_expr2 THEN return_expr2  
          WHEN comparison_exprn THEN return_exprn  
          ELSE else_expr]  
END
```

50

Exemple CASE

```
SELECT last_name, job_id, salary,
       CASE job_id WHEN 'IT_PROG' THEN 1.10*salary
                   WHEN 'ST_CLERK' THEN 1.15*salary
                   WHEN 'SA_REP' THEN 1.20*salary
       ELSE salary END "REVISED_SALARY"
FROM   employees;
```

51

Fonction DECODE

Syntaxe

```
DECODE(col|expression, search1, result1
      [, search2, result2,...,]
      [, default])
```

52

Exemple DECODE

```
SELECT last_name, job_id, salary,
       DECODE(job_id, 'IT_PROG', 1.10*salary,
               'ST_CLERK', 1.15*salary,
               'SA_REP', 1.20*salary,
               salary)
       "REVISED_SALARY"
FROM   employees;
```

53

Exemple Oracle DECODE

Afficher le taux d'imposition applicable pour chaque employé du département 80 :

```
SELECT last_name, salary,
       DECODE (TRUNC(salary/2000, 0),
              0, 0.00,
              1, 0.09,
              2, 0.20,
              3, 0.30,
              4, 0.40,
              5, 0.42,
              6, 0.44,
              0.45) TAX_RATE
FROM   employees
WHERE  department_id = 80;
```

54

Langage PL/SQL

55

PL/SQL

Produits Secteurs d'activité Documentation Clients Partenaires Développeurs Entreprise

Database / Technologies / Développement d'applications Database / PL/SQL



PL/SQL hérite de la robustesse, de la sécurité et de la portabilité d'Oracle Database

PL/SQL est un langage procédural spécifiquement conçu pour accepter les instructions SQL au sein de sa syntaxe. Les unités du programme PL/SQL sont compilées par le serveur Oracle Database et stockées dans la base de données. Au moment de l'exécution, les deux langages PL/SQL et SQL s'exécutent au sein du même processus de serveur, pour une efficacité optimale. PL/SQL hérite automatiquement de la robustesse, de la sécurité et de la portabilité d'Oracle Database.

Essayer PL/SQL

56

Langage procédural PL/SQL

Supporte SQL

Syntaxe ADA / Pascal

57

Langage procédural PL/SQL

Bloc d'instruction structuré

Définition bloc anonyme:

```
[DECLARE
/* Declarations des variables, types, s/s prog. Locaux */
BEGIN
    -- instructions: corps du bloc
    [EXCEPTION /* gestion exception*/ ]
END;
```

Commentaire: -- ou /* */

Conventions: mot clé en majuscules

58

Exemple

```
DECLARE
  v_ename VARCHAR2(5);
BEGIN
  SELECT ename INTO v_ename
    FROM emp
   WHERE empno = 1234;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    ...
END;
/
```

59

Langage procédural PL/SQL

```
DECLARE
  DECLARE nom_variable type_variable;

  %ROWTYPE
  DECLARE tuple table_existante%ROWTYPE;

  %TYPE
  DECLARE nom_variable table.attribut%TYPE;
```

60

Langage procédural PL/SQL

Assignation: **identifiant := expression;**

SELECT INTO %FOUND %NOTFOUND

- ex. SELECT INTO variable * FROM R WHERE R.A=variable2;
- ex. SELECT att1, att2, ... attn INTO variable1, variable2, ...variablen FROM ... WHERE ... (1
seul type retourné avec 1 ou plus. Colonnes)

Structure de contrôle

```
if condition
then
  ...
else
  ...
end if;
```

61

Langage procédural PL/SQL

Structures de boucle

<pre>loop ... end loop; ⇒ if ... exit; end if; ⇒ exit when condition</pre>	<pre>while condition loop ... end loop;</pre>	<pre>for variable in debut .. fin loop ... end loop;</pre>
--	---	--

62

Remarque

Possibilité de nommer les LOOP
LOOP

```
...  
-- exit nom_loop when  
...  
END LOOP nom_loop;
```

63

Labels

Déclaration

```
<< Nom_Label >>
```

Accès par un goto

```
goto Nom_Label
```

64

Curseur

Zone privée de SQL pour exécuter les commandes et stocker les informations

Lié au concept de transaction

Permet de sélectionner un ensemble de n-uplets pour les manipuler un à un <=> pointeur

65

Remarque préliminaire : Curseur SQL

Syntaxe Curseur SQL :

```
DECLARE cursor_name [ BINARY ] [ INSENSITIVE ] [ SCROLL ]  
CURSOR FOR query  
[ FOR { READ ONLY | UPDATE [ OF column [, ...] ] }
```

Retrouver les n-uplets : FETCH

Syntaxe FETCH

```
FETCH [ direction ] [ count ] { IN | FROM } cursor  
FETCH [ FORWARD | BACKWARD | RELATIVE ] [ # | ALL | NEXT | PRIOR ] { IN | FROM } cursor
```

66

Utilisation d'un Curseur

4 étapes :

1. Ouverture
2. Manipulation
3. Atteinte des enregistrements
4. Fermeture du curseur

67

Curseur en PL/SQL

Curseur Explicite: Ordre SELECT explicitement défini dans section DECLARE

Déclaration:

```
DECLARE
  CURSOR nom_curseur [(param1, ...)]
  [RETURN specifications_retour]
  IS
    SELECT...
    FROM...
    WHERE...
    [FOR UPDATE [OF [liste_col]]][NOWAIT];
```

68

Curseur PL/SQL FOR...LOOP

Curseur avec ouverture, fermeture et accès implicites avec boucle FOR

Syntaxe:

```
DECLARE
  CURSOR C1 IS SELECT ...
  Tuple table%ROWTYPE      ← optionnel !!!
BEGIN
  For Tuple in C1
  Loop
    ...
  End loop;
End;
```

69

Exemples

```
DECLARE CURSOR c1 IS SELECT empno FROM EMP;
```

```
DECLARE CURSOR C2(param IN NUMBER) IS SELECT ename FROM EMP WHERE EMPNO=param;
```

```
CURSOR c3 RETURN EMP%ROWTYPE IS SELECT * FROM EMP WHERE DEPTNO=10
```

70

Curseur en PL/SQL

Ouverture du curseur (OPEN)

```
OPEN nom_curseur [(arg1, ...)];
```

Manipulation des enregistrements (FETCH)

```
FETCH nom_curseur INTO cible;
```

Atteinte des enregistrements

Fermeture du curseur (CLOSE)

```
CLOSE nom_curseur;
```

71

Curseur en PL/SQL

Nom_curseur%ROWCOUNT Nombre de lignes du curseur à un instant donné

Nom_curseur%FOUND Retourne TRUE si un enregistrement est extrait.

Nom_curseur%NOTFOUND Retourne TRUE si aucun enregistrement n'est extrait

Nom_curseur%ISOPEN Retourne TRUE si un curseur est ouvert

72

Exemple

DECLARE

```
CURSOR c1 IS SELECT NoEmp, NomE, Poste FROM emp ;
```

```
OPEN curseur                    Ensemble actif
```

```
FETCH curseur            --100   JEAN           MGR
```

```
                         --101   PIERRE          ING
```

```
                         --102   ALICE           SECR
```

```
                         --103   ADAM           TECH
```

```
CLOSE curseur
```

73

Curseur avec paramètres

Ouverture et passage de paramètres:

```
CURSOR c1 (liste_paramètres) IS ...
```

```
OPEN c1(valeur_paramètres);
```

Exemple :

```
CURSOR c1 (nom_emp text , comm integer) IS ...
```

```
OPEN c1('Jean', 150);
```

74

Exemple

Recherche de tuple

```
LOOP
    FETCH c1 INTO mon_rec;
    EXIT WHEN c1%NOT FOUND;
    -- traitement des données

END LOOP;
CLOSE c1;
```

75

CURRENT OF

Utilisé dans le cas d'un curseur FOR UPDATE

Permet d'accéder directement à l'enregistrement courant pour une modification ou une mäj

76

Exemple Oracle

```
DROP TABLE emp;

CREATE TABLE emp AS SELECT * FROM
employees;

DECLARE
    CURSOR c1 IS SELECT * FROM emp FOR
UPDATE OF salary ORDER BY employee_id;
    emp_rec emp%ROWTYPE;

BEGIN
    OPEN c1;
    LOOP
        FETCH c1 INTO emp_rec;
        EXIT WHEN c1%NOT FOUND;
        DBMS_OUTPUT.PUT_LINE('emp_rec.employee_id = '
|| TO_CHAR(emp_rec.employee_id));
        UPDATE emp
        SET salary = salary * 1.05
        WHERE employee_id = 105;
        COMMIT; -- releases locks
    END LOOP;
END; /
```

77

Curseur REF

Syntaxe

TYPE ref_type_name IS REF CURSOR[RETURN return_type];

- ref_type_name: 1 type spécifié dans les déclarations
- return_type : représente 1 type record (enregistrement)
- return_type: indique 1 curseur

Exemple

```
DECLARE
    TYPE rt_cust IS REF CURSOR
    RETURN customers%ROWTYPE;
...
```

78

Etape 1

Déclaration d'une variable du type CURSEUR
CURSOR_VARIABLE_NAME REF_TYPE_NAME

```
DECLARE
  TYPE rt_cust IS REF CURSOR RETURN customers%ROWTYPE;
  cv_cust rt_cust;
```

79

Etape 1: options possibles

Utiliser
%TYPE
%ROWTYPE

Spécifier un RECORD utilisateur dans le RETURN

Déclarer la variable de type Curseur comme paramètre d'une procédure ou fonction

80

Etape 2: Ouverture

Associer le CURSEUR avec une requête qui retourne un tuple (multicolonne)

Exécuter la requête

Identifier le résultat

```
OPEN cursor_variable_name
FOR select_statement
```

81

Etape 3: Fetch

```
FETCH cursor_variable_name INTO variable_name1 [,variable_name2, . . .] | record_name;
```

Le type de retour de la variable de type curseur doit être compatible avec les variables nommées dans la clause INTO du FETCH

82

Etape 4: Fermeture

Désactiver la variable de type curseur

```
CLOSE cursor_variable_name ;
```

83

Curseur prédéfini SYS_REFCURSOR

SYS_REFCURSOR est un REF CURSOR qui permet d'associer n'importe quel ensemble de résultats, cela permet de :

- Déclarer une variable de curseur dans une procédure stockée Oracle ou une fonction
- Passer des curseurs depuis et vers une procédure ou une fonction stockée Oracle

85

Exemple Oracle

```
CREATE OR REPLACE PROCEDURE REFCUR (p_num IN NUMBER)
IS
    refcur sys_refcursor;
    empno      emp.empno%TYPE;    ename      emp.ename%TYPE;
BEGIN
    IF p_num = 1 THEN
        OPEN refcur FOR SELECT empno, ename FROM emp;
        DBMS_OUTPUT.PUT_LINE('Employee#      Name');
        DBMS_OUTPUT.PUT_LINE('-----      -');
        LOOP
            FETCH refcur INTO empno, ename;
            EXIT WHEN refcur%NOTFOUND;
            DBMS_OUTPUT.PUT_LINE(empno || '      ' || ename);
        END LOOP;
    END IF;
END;
```

86

Exemple Oracle

```
ELSE
    OPEN refcur FOR SELECT empno, ename FROM emp WHERE deptno = 30;
    DBMS_OUTPUT.PUT_LINE('Employee#      Name');
    DBMS_OUTPUT.PUT_LINE('-----      -');
    LOOP
        FETCH refcur INTO empno, ename;
        EXIT WHEN refcur%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(empno || '      ' || ename);
    END LOOP;
END IF;
CLOSE refcur;
END;
/
```

87

Curseur implicite

Généré par le noyau lui-même

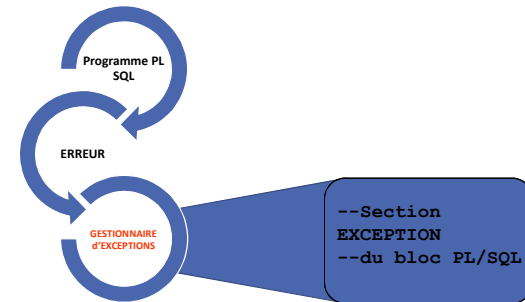
Ordre SELECT, INSERT, UPDATE, SELECT INTO (à une seule ligne)

Attributs possibles:

SQL%ROWCOUNT	Nombre de lignes affectées par la dernière instruction SQL
SQL%FOUND	Retourne TRUE si la dernière instruction SQL a retourné une ou plusieurs lignes
SQL%NOTFOUND	Retourne TRUE si la dernière instruction SQL n'a retourné aucune ligne
SQL%ISOPEN	Détermine si un curseur est ouvert FALSE toujours car curseurs implicites fermés systématiquement après exécution

88

Gestion des exceptions



89

Syntaxe

```
EXCEPTION
WHEN exception1 [OR exception2...] THEN
statement1;
statement2;
...
[WHEN exception3 [OR exception4...] THEN
statement1;
statement2;
...]
[WHEN OTHERS THEN
statement1;
statement2;
...]
```

90

Exceptions

Prédéfinies par Oracle exemple:

- NO_DATA_FOUND
- TOO_MANY_ROWS
- INVALID_CURSOR
- ZERO_DIVIDE
- DUP_VAL_ON_INDEX

91

Erreur ORACLE

Exception Name	Error Code	Exception Name	Error Code
ACCESS_INTO_NULL	-6530	PROGRAM_ERROR	-6501
CASE_NOT_FOUND	-6592	ROWTYPE_MISMATCH	-6504
COLLECTION_IS_NULL	-6531	SELF_IS_NULL	-30625
CURSOR_ALREADY_OPEN	-6511	STORAGE_ERROR	-6500
DUP_VAL_ON_INDEX	-1	SUBSCRIPT_BEYOND_COUNT	-6533
INVALID_CURSOR	-1001	SUBSCRIPT_OUTSIDE_LIMIT	-6532
INVALID_NUMBER	-1722	SYS_INVALID_ROWID	-1410
LOGIN_DENIED	-1017	TIMEOUT_ON_RESOURCE	-51
NO_DATA_FOUND	+100	TOO_MANY_ROWS	-1422
NO_DATA_NEEDED	-6548	VALUE_ERROR	-6502
NOT_LOGGED_ON	-1012	ZERO_DIVIDE	-1476

92

Exception non prédéfinie

Nommer l'exception

Code PRAGMA EXCEPTION_INIT : Associe une exception utilisateur (nom) avec un code erreur

Manipuler l'exception levée

93

Exception non prédéfinie : Syntaxe

```
[DECLARE]
nom_exception EXCEPTION;
PRAGMA EXCEPTION_INIT (nom_exception, code_erreur);
...
BEGIN
...
EXCEPTION
WHEN nom_exception ...
END;/
```

94

Exceptions utilisateur

Nommer l'exception

Lever l'exception avec RAISE

Manipuler l'exception levée

95

Exceptions utilisateur : Syntaxe

```
-- section declaration
[DECLARE]
    nom_exception EXCEPTION ...
BEGIN ...
-- test conditionnel
IF condition THEN
    RAISE nom_exception
ENDIF
...
EXCEPTION
    WHEN nom_exception ...
END;
/
```

96

Exemple Oracle

```
CREATE PROCEDURE account_status ( due_date DATE, today DATE) AUTHID DEFINER
IS
    past_due EXCEPTION; -- declare exception
BEGIN
    IF due_date < today THEN
        RAISE past_due; -- explicitly raise exception
    END IF;
EXCEPTION
    WHEN past_due THEN -- handle exception
        DBMS_OUTPUT.PUT_LINE ('Account past due.');
```

97

Exemple Oracle

```
DROP TABLE t; CREATE TABLE t (c NUMBER);
CREATE PROCEDURE p (n NUMBER) AUTHID DEFINER IS default_number NUMBER := 0;
BEGIN
    IF n < 0 THEN RAISE INVALID_NUMBER; -- raise explicitly
    ELSE
        INSERT INTO t VALUES(TO_NUMBER('100.00', '9G999')); -- raise implicitly
    END IF;
EXCEPTION
    WHEN INVALID_NUMBER THEN
        DBMS_OUTPUT.PUT_LINE('Substituting default value for invalid number.');
```

98

Interception Erreur Oracle

```
BEGIN ...
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        statement1;
        statement2;
    WHEN TOO_MANY_ROWS THEN
        statement1;
    WHEN OTHERS THEN
        statement1;
        statement2;
        statement3;
```

99

Exemple Oracle

```
DECLARE
    stock_price  NUMBER := 9.73; net_earnings  NUMBER := 0;
    pe_ratio     NUMBER;
BEGIN
    pe_ratio := stock_price / net_earnings; -- raises ZERO_DIVIDE
exception
    DBMS_OUTPUT.PUT_LINE('Price/earnings ratio = ' || pe_ratio);
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Company had zero earnings.');
```

```
pe_ratio := NULL;    END;    /
```

100

RAISE_APPLICATION_ERROR

Permet de publier des messages d'erreurs définis par l'utilisateur depuis un bloc PL/SQL:

- annoncer des erreurs à une application
- éviter de retourner des exceptions non définies

Défini dans le package DBMS_STANDARD

Utilisable

- Section exécutable
- Section exception

101

RAISE_APPLICATION_ERROR

Syntaxe :

RAISE_APPLICATION_ERROR (error_number, message [{TRUE|FALSE}]);

- error_number est le numéro d'exception spécifié par le développeur qui doit se trouver entre -20000 et -20999.
- message est le message d'erreur associée à l'exception et spécifié par le développeur. Il s'agit d'une chaîne de caractères avec une taille maximale de 2048 octets.
- TRUE: on place l'erreur dans la pile sinon remplace les erreurs précédentes

102

Exemple Oracle

```
CREATE PROCEDURE account_status (due_date DATE, today DATE) AUTHID
DEFINER
IS
BEGIN
    IF due_date < today THEN                -- explicitly raise exception
        RAISE_APPLICATION_ERROR(-20000, 'Account past due.');
```

```
END IF;
END;
```

```
/
```

103

Exemple Oracle

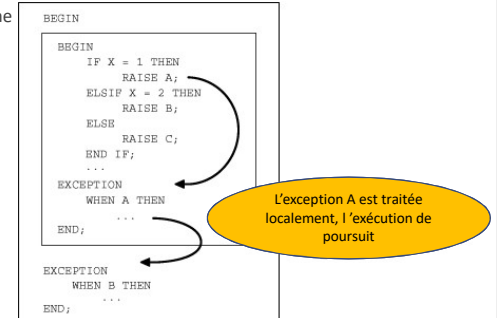
```
...  
BEGIN  
...  
    DELETE FROM emp  
    WHERE mgr = v_mgr;  
...  
    EXCEPTION  
        WHEN NO_DATA_FOUND THEN  
            RAISE_APPLICATION_ERROR (-20201, 'This is not a valid  
manager');  
END;
```

104

Propagation

Si l'exception est traitée dans le bloc, elle ne se propage pas

Exemple Oracle

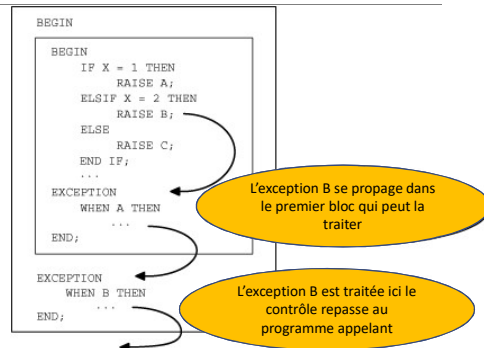


105

Propagation

Si l'exception n'est pas traitée dans le bloc, elle se propage

Exemple Oracle

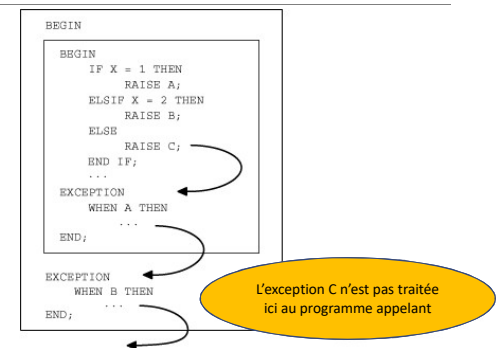


106

Propagation

Si l'exception n'est pas traitée dans le bloc, elle se propage

Exemple Oracle



107

Exemple Oracle

```
CREATE OR REPLACE PROCEDURE p
AUTHID DEFINER AS BEGIN
    DECLARE
        past_due    EXCEPTION;
        PRAGMA EXCEPTION_INIT
        (past_due, -4910);
        due_date    DATE :=
        trunc(SYSDATE) - 1;
        todays_date DATE :=
        trunc(SYSDATE);
    BEGIN
        IF due_date < todays_date THEN
            RAISE past_due;
        END IF;
    END;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END;
/
```

108

Exemple Oracle

```
BEGIN
    DECLARE
        past_due    EXCEPTION;
        due_date    DATE :=
        trunc(SYSDATE) - 1;
        todays_date DATE :=
        trunc(SYSDATE);
    BEGIN
        IF due_date < todays_date THEN
            RAISE past_due;
        END IF;
    END;
END IF;
```

109

Procédures et Fonctions

Procédure

```
CREATE [OR REPLACE] PROCEDURE nom_proc
(param1 [MODE1] type1 [[DEFAULT]:=] valeur1, ...)
IS | AS
-- bloc PL/SQL
END [Nom_proc]
```

112

Procédures et fonctions

Mode: IN | OUT | INOUT

Exécution:

- EXECUTE Nom_proc(...); -- sous SQL*Plus
- Nécessite des variables
- Passage des paramètres avec :nom_variable
- Nom_proc(...) -- depuis un bloc

Passage paramètres:

- Par position: selon ordre des paramètres
- Par association de nom: selon la syntaxe : *nom_parametre* => *valeur*
- Par combinaison: 1^{ers} par position le reste par a.de n.

113

Procédures et Fonctions

Fonctions

```
CREATE [OR REPLACE] FUNCTION nom_fct  
(param1 [MODE1] type1 [[DEFAULT|:=] valeur1], ...)  
RETURN TYPE  
IS | AS  
-- bloc PL/SQL  
END [Nom_proc]
```

114

Sous-programmes

Possibles en déclarant les procédures avant le bloc IS

115

Sources

Table USER_SOURCE

- Sous-programmes dont on est le propriétaire

```
SELECT text  
FROM user_source  
WHERE name = 'ADD_DEPARTMENT' AND type = 'PROCEDURE'  
ORDER BY line;
```

Table ALL_SOURCE

- Procédures qui appartiennent à d'autres et sur lesquelles on a un droit d'exécution (EXECUTE privilege)

Table USER_OBJECTS

```
SELECT object_name  
FROM user_objects  
WHERE object_type= 'PROCEDURE';
```

116

Procédure stockée

Appel via une variable d'environnement

```
VARIABLE salary NUMBER  
EXECUTE :salary := get_sal(100)
```

Appel via un bloc PL/SQL

```
DECLARE  
    sal employees.salary%type;  
BEGIN  
    sal := get_sal(100);    ...  
END;
```

117

Procédure stockée

Appel via un autre sous-programme

```
EXECUTE dbms_output.put_line(get_sal(100))
```

Appel via un ordre SQL

```
SELECT job_id, get_sal(employee_id) FROM employees;
```

118

Visualisation Code Fonction

USER_SOURCE ou ALL_SOURCE

```
SELECT text
FROM user_source
WHERE type = 'FUNCTION'
ORDER BY line;
```

USER_OBJECTS

```
SELECT object_name
FROM user_objects
WHERE object_type = 'FUNCTION';
```

119

Packages

120

Packages

Un groupe contenant

- Types
- Eléments: variables, curseurs
- Sous-programmes: fonction, proc.

Ecrits en PL/SQL

121

Packages Standards Oracle

DBMS_ALERT
DBMS_LOCK
DBMS_SESSION
DBMS_OUTPUT
HTP
UTL_FILE
UTL_MAIL
DBMS_SCHEDULER

122

Package DBMS_OUTPUT

Permet d'envoyer des informations à partir

- Sous-programmes
- Triggers

PUT et PUT_LINE: texte placé dans le Buffer

GET_LINE et GET_LINES lisent le buffer

Utiliser SET SERVEROUTPUT ON pour afficher les messages

123

Package UTL_FILE

Permet de lire et écrire des fichiers textes à partir de PL/SQL

- Equivalent OS stream
- Nécessite un CREATE_DIRECTORY ou utilisation du paramètre de la BD *utl_file_dir*



124

Packages

Encapsulation des procédures, variables et types de données

Séparation de la spécification et le corps du package

Déclaration de procédure « public » et « private »

Définition de variables persistantes

Amélioration des performances

125

Packages

Composé d'une partie

- Spécification (public) :
 - Définit les spécifications disponibles pour les applications
 - Y sont déclarés les types, les variables, les constantes, les exceptions, les curseurs et les sous-programmes utilisables dans le package
- Corps (private) :
 - Définit les curseurs et les sous-programmes spécifiés dans la partie spécification

126

Syntaxe Package

Spécifications

```
CREATE [OR REPLACE] PACKAGE nom_package
```

```
IS | AS
```

```
-- déclaration variables
```

```
-- déclaration des types
```

```
-- déclaration des cstes, curseurs, public
```

```
-- spécifications des s/s prog.
```

```
END nom_package;
```

127

Syntaxe Package

Corps

```
CREATE [OR REPLACE] PACKAGE BODY nom_package
```

```
IS | AS
```

```
-- code du package
```

```
END nom_package;
```

128

Package

Constantes/variables globales

- Durée de vie des structures de données: pdt la durée de la session
- Donnée déclarée dans le corps: persiste pendant la session
- Donnée déclarée dans spécifications: donnée publique

Curseur ouvert dans une procédure packagée reste ouvert pendant la session

Les variables peuvent exister au-delà des transactions mais non partagées par plusieurs sessions
sinon utiliser package DBMS_PIPE

129

Constantes variables globales

Déclarées dans les spécifications d'un package

- Variables globales
- Corps inutile!

130

Exemple Oracle

```
CREATE OR REPLACE PACKAGE global_consts IS
  mile_2_kilo CONSTANT    NUMBER := 1.6093;
  kilo_2_mile CONSTANT    NUMBER := 0.6214;
  yard_2_meter CONSTANT   NUMBER := 0.9144;
  meter_2_yard CONSTANT   NUMBER := 1.0936;
END global_consts;

BEGIN
  DBMS_OUTPUT.PUT_LINE('20 miles = ' || 20 *
    global_consts.mile_2_kilo || ' km');
END;
```

131

Exemple Oracle (suite)

```
CREATE FUNCTION mtr2yrd(m NUMBER) RETURN NUMBER IS
BEGIN
  RETURN (m * global_consts.meter_2_yard);
END mtr2yrd;
/
EXECUTE DBMS_OUTPUT.PUT_LINE(mtr2yrd(1))
```

132

Exemple

Spécification

```
CREATE OR REPLACE PACKAGE nom_package IS
  CURSOR nom_c1([param1, ...]) RETURN nom_table%ROWTYPE
END nom_package;
```

Corps

```
CREATE OR REPLACE PACKAGE BODY nom_package IS
  CURSOR nom_c1([param1,...]) RETURN nom_table%ROWTYPE IS
  SELECT .. FROM ... WHERE ...
  ...
```

133

Manipulation des packages

Procédure publique : accessible via EXECUTE sous Prompt : EXECUTE nom_package.nom_proc (parametres)

Dans le package, inutile de nommer le package

Surcharge possible

Sous-type utilisateur: SUBTYPE nom_ss_type IS type_de_base;

134

Curseur packagé

Défini comme curseur interne (cf. exemple suivant)

Défini dans les spécifications du package

Comme un curseur classique mais de type REF

Seulement avec un entête sans la requête: nécessite un RETURN qui précise les éléments renvoyés par le FETCH

135

Exemple

--spécifications

CREATE OR REPLACE PACKAGE pkgemployecursor IS

 PROCEDURE afficheremployes(pdeptno IN NUMBER);

END pkgemployecursor;

/

136

Exemple

```
-- corps
CREATE OR REPLACE PACKAGE BODY pkgemployecursor IS
    PROCEDURE afficheremployes(pdeptno IN NUMBER) IS
        CURSOR empcursor IS
            SELECT empno, ename, deptno FROM emp
            WHERE deptno = pdeptno
            ORDER BY ename;
        vempno emp.empno%TYPE; vename emp.ename%TYPE;
        vdeptno emp.deptno%TYPE;
    BEGIN
        DBMSOUTPUT.PUTLINE('Liste des employés du département '
        || pdeptno);
        DBMSOUTPUT.PUTLINE('-----');
        OPEN empcursor;
        LOOP
            FETCH empcursor INTO vempno, vename, vdeptno;
            EXIT WHEN empcursor%NOTFOUND;
            DBMSOUTPUT.PUTLINE(vempno || ' - ' || vename || ' -
            Dept: ' || vdeptno);
        END LOOP;
        CLOSE empcursor;
    END afficheremployes;
END pkgemploye_cursor;
```

137

Exemple 2

```
-- specif
CREATE OR REPLACE PACKAGE pkg_employe IS
TYPE emp_cursor_type IS REF CURSOR;
FUNCTION get_employes_by_dept(p_deptno IN NUMBER) RETURN emp_cursor_type;
END pkg_employe;
```

138

Exemple 2

```
CREATE OR REPLACE PACKAGE BODY pkg_employe IS
FUNCTION get_employes_by_dept(p_deptno IN NUMBER) RETURN emp_cursor_type IS
emp_cursor emp_cursor_type;
BEGIN
OPEN emp_cursor FOR
SELECT empno, ename, deptno
FROM emp
WHERE deptno = p_deptno
ORDER BY ename;
RETURN emp_cursor;
END get_employes_by_dept;
END pkg_employe;
```

139

Exemple 2

```
DECLARE
rc pkg_employe.emp_cursor_type; v_empno emp.empno%TYPE;
v_ename emp.ename%TYPE; v_deptno emp.deptno%TYPE;
BEGIN
rc := pkg_employe.get_employes_by_dept(20);
LOOP
FETCH rc INTO v_empno, v_ename, v_deptno;
EXIT WHEN rc%NOTFOUND;
DBMS_OUTPUT.PUT_LINE(v_empno || ' - ' || v_ename || ' - Dept: ' || v_deptno);
END LOOP;
CLOSE rc;
END;
```

140

Exemple avec SYS_REFCURSOR

```
CREATE OR REPLACE PACKAGE pkgemployees IS
PROCEDURE getemployees(pnum IN NUMBER, pcursor OUT SYS_REFCURSOR);
END pkgemployees;
--corps
CREATE OR REPLACE PACKAGE BODY pkg_employees IS
PROCEDURE get_employees(p_num IN NUMBER, p_cursor OUT SYS_REFCURSOR) IS
BEGIN
IF p_num = 1 THEN
OPEN p_cursor FOR SELECT empno, ename FROM emp;
ELSE
OPEN p_cursor FOR SELECT empno, ename FROM emp WHERE deptno = 30;
END IF;
END get_employees;
END pkg_employees;
```

141

Exemple avec SYS_REFCURSOR

```
DECLARE
  rc SYS_REFCURSOR;
  v_empno emp.empno%TYPE;
  v_ename emp.ename%TYPE;
BEGIN
  pkg_employees.get_employees(1, rc);
  LOOP
    FETCH rc INTO v_empno, v_ename;
    EXIT WHEN rc%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_empno || ' - ' || v_ename);
  END LOOP;
  CLOSE rc;
END;
/
```

142

Vue package

Tables USER_SOURCE et ALL_SOURCE

◦ Spécifications

```
SELECT text
FROM user_source
WHERE name = 'COMM_PKG' AND type = 'PACKAGE';
```

◦ Body

```
SELECT text
FROM user_source
WHERE name = 'COMM_PKG' AND type = 'PACKAGE BODY';
```

143

Surcharge

Paramètres différents (ordre ou nombre ou type)

Fonctionne avec

- Sous-programmes de packages
- Sous-programmes locaux

144

Exemple

Spécification du package

```
CREATE OR REPLACE PACKAGE dept_pkg IS
  PROCEDURE add_department (deptno NUMBER,
                           name VARCHAR2 := 'unknown',
                           loc NUMBER := 1700);
  PROCEDURE add_department (name VARCHAR2 := 'unknown',
                           loc NUMBER := 1700);
END dept_pkg;
/
```

145

Exemple

Corps du package

```
CREATE OR REPLACE PACKAGE BODY dept_pkg IS
  PROCEDURE add_department (deptno NUMBER, name VARCHAR2 := 'unknown',
                           loc NUMBER := 1700) IS
  BEGIN
    INSERT INTO departments (department_id, department_name, location_id)
      VALUES (deptno, name, loc);
  END add_department;

  PROCEDURE add_department (name VARCHAR2 := 'unknown', loc NUMBER := 1700) IS
  BEGIN
    INSERT INTO departments (department_id, department_name, location_id)
      VALUES (departments_seq.NEXTVAL, name, loc);
  END add_department;
END dept_pkg; /
```

146

Surcharge du package STANDARD

Définit l'environnement PL/SQL et les fonctions

Exemple la fonction TO_CHAR

```
FUNCTION TO_CHAR (p1 DATE) RETURN VARCHAR2;
FUNCTION TO_CHAR (p2 NUMBER) RETURN VARCHAR2;
FUNCTION TO_CHAR (p1 DATE, p2 VARCHAR2) RETURN VARCHAR2;
FUNCTION TO_CHAR (p1 NUMBER, p2 VARCHAR2) RETURN VARCHAR2;
```

Surcharge

- valable localement

147

Déclaration postérieure

Exemple de pb

```
CREATE OR REPLACE PACKAGE BODY forward_pkg IS
  PROCEDURE award_bonus(...) IS
  BEGIN
    calc_rating (...); --illegal reference
  END;

  PROCEDURE calc_rating (...) IS
  BEGIN
    ...
  END;
END forward_pkg;
/
```

148

Déclaration postérieure

Exemple

```
CREATE OR REPLACE PACKAGE BODY forward_pkg IS
  PROCEDURE calc_rating (...); --forward declaration
  -- subprograms defined in alphabetical order

  PROCEDURE award_bonus(...) IS
  BEGIN
    calc_rating (...); --reference resolved
  END;

  PROCEDURE calc_rating (...) IS --implementation
  BEGIN
    ...
  END;
END forward_pkg; /
```

149

Package

Bloc d'initialisation

- Exécuté une fois
- Initialisation variables public et privée

```
CREATE OR REPLACE PACKAGE taxes IS
  tax NUMBER;
  ...    -- Declare all public procedures/functions
END taxes; /
CREATE OR REPLACE PACKAGE BODY taxes IS
  -- Declare all private variables
  -- Define public/private procedures/functions
BEGIN
  SELECT rate_value INTO tax
  FROM   tax_rates
  WHERE  rate_name = 'TAX';
END taxes; /
```

150

Utilisation des packages

Possible dans des requêtes SQL ou du LMD

- Fonctions appelées ne doivent pas à partir
 - D'une requête, finir la transaction courante, faire un savepoint ou faire un rollback vers un savepoint
 - D'une requête, modifier la base si la requête est parallélisée
 - D'un ordre LMD lire ou modifier la table modifiée par la requête

151

Exemple d'utilisation

```
CREATE OR REPLACE PACKAGE taxes_pkg IS
  FUNCTION tax (value IN NUMBER) RETURN NUMBER;
END taxes_pkg;
/
CREATE OR REPLACE PACKAGE BODY taxes_pkg IS
  FUNCTION tax (value IN NUMBER) RETURN NUMBER IS
    rate NUMBER := 0.08;
  BEGIN
    RETURN (value * rate);
  END tax;
END taxes_pkg;
-- exemple utilisation
SELECT
  taxes_pkg.tax(salary) AS tax,  salary,  last_name
FROM employees;
```

152

Persistance: Etat du package

Initialisé quand il est chargé pour la première fois

Persistant lors de la session utilisateur

- Sauvegardé dans la UGA (User Global Area)
- Unique à chaque session
- Modifié éventuellement par les sous-programmes
- Pendant la vie d'un sous-programmes si utilisation de la directive PRAGMA SERIALLY_REUSABLE dans les spécifications du package

153

Curseur packagé : Exemple

```
CREATE OR REPLACE PACKAGE BODY
curs_pkg IS
CURSOR c IS SELECT employee_id FROM
employees;
PROCEDURE open IS
BEGIN
IF NOT c%ISOPEN THEN OPEN c;
END IF;
END open;
FUNCTION next (n NUMBER := 1) RETURN
BOOLEAN IS
emp_id employees.employee_id%TYPE;
BEGIN
FOR count IN 1 .. n LOOP
FETCH c INTO emp_id;
EXIT WHEN c%NOTFOUND;
DBMS_OUTPUT.PUT_LINE('Id: ' ||
emp_id);
END LOOP;
RETURN c%FOUND;
END next;
PROCEDURE close IS
BEGIN
IF c%ISOPEN THEN CLOSE c;
END IF;
END close;
END curs_pkg;
```

154

Curseur packagé

Exemple Utilisation

```
SET SERVEROUTPUT ON
EXECUTE curs_pkg.open
DECLARE
more BOOLEAN := curs_pkg.next(3);
BEGIN
IF NOT more THEN
curs_pkg.close;
END IF;
END;
/
RUN
-- repeats execution on the anonymous block
EXECUTE curs_pkg.close
```

155

SQL Wrapper

Conversion du code PL/SQL en code exécutable

Permet

- Indépendance de plateforme
- Chargement dynamique
- Check des dépendances

156

SQL Wrapper

Syntaxe

```
WRAP INAME=input_file_name [ONAME=output_file_name]
```

- INAME paramètre obligatoire
- Extension .sql par défaut
- ONAME optionnel
- Extension en sortie .plb par défaut

```
WRAP INAME=demo_04_hello.sql
WRAP INAME=demo_04_hello
```

157

SQL Wrapper : Résultats

Avant

```
CREATE PACKAGE banking IS
  min_bal := 100;
  no_funds EXCEPTION;
  ...
END banking;
/
```

Après

```
CREATE PACKAGE banking
wrapped
012abc463e ...
/
```

158

SQL Wrapper

Sur le corps du package

PAS sur les spécifications du package

A exécuter après chaque modification sur le code PL/SQL

159

Les TRIGGERS

160

Les triggers

Procédures compilées, présente dans le dictionnaire, s'exécutant automatiquement lorsqu'un événement spécifié se produit

Un Mécanisme de contrôle extensible

Utilisation des triggers:

- validation données saisies ou entrées
- création d'un fichier de log (audit)
- contraintes d'intégrités complexes

161

Les triggers

Modèle élémentaire :

- Règles Événement / Condition / Action
- Détection Événement, si Condition est vérifiée alors Déclenchement Action
- Action: un ou plusieurs ordres SQL, SQL procédural qui s'applique à la base

Se déclenchent

- sur ordre LMD
 - 1 fois et une seule → trigger ORDRE
 - Pour chaque ligne traitée → trigger LIGNE (⇒FOR EACH ROW)
- Sur DBA

162

Les triggers

Événement

CONDITION

ACTION

163

Les triggers

La syntaxe générale est:

```
CREATE [OR REPLACE] TRIGGER nom_du_trigger
evenement ON nom_table [FOR EACH ROW]
[WHEN (condition)]
-- instructions PL/SQL
END [nom_du_trigger]
```

164

Les triggers

Évènement :

1 interposition

- BEFORE
- AFTER
- INSTEAD OF

et un ordre sur une table

- UPDATE
- INSERT
- DELETE
- ou une combinaison de ces derniers avec un OR

165

Les triggers

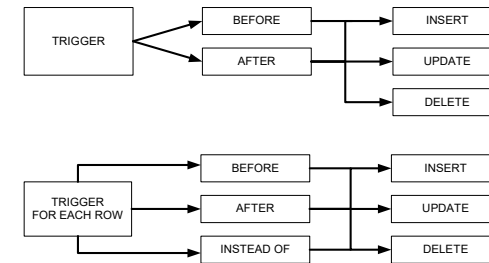
La **condition** restreint l'application du trigger

Le concepteur a le choix entre spécifier l'action à exécuter soit

- Une fois, pour chaque tuple modifié/inséré/supprimé
- **FOR EACH ROW** indique que les instructions doivent s'appliquer à chaque insertion/modification/suppression d'un tuple
- Une fois pour tous les tuples modifiés lors d'une seule opération
- alors que si non EACH ROW ne le fait qu'une fois par évènement déclenchant

166

Les triggers



167

Les triggers

Par exemple, si l'on veut faire qu'une table de statistiques sur les employés se tienne à jour automatiquement à chaque introduction d'une donnée sur EMP, on peut créer un trigger pour mettre à jour cette table.

168

Les triggers

Pour un trigger opérant sur les lignes, il existe 2 variables utilisables dites de transition

:old

:new

Les modifications font passer de "old" à "new"

- on peut faire référence à ces deux valeurs

Cette variable avant n'a pas de sens dans un INSERT

Cette variable après n'a pas de sens dans un DELETE

169

Les triggers

Les insertions créent de nouveaux tuples

- impossible de faire référence à l'ancienne valeur

Avec les suppressions, on ne peut faire référence aux nouvelles valeurs.

Les nouvelles/anciennes valeurs ne peuvent être accédées qu'en mode ligne (cas d'un FOR EACH ROW)

170

Les triggers

WHEN (attribut condition valeur)

- attribut: une colonne de la ligne avant (old.attribut) ou après (new.attribut) l'événement
- condition: =, <>, <>, BETWEEN, IN
- valeur: Utilisation d'un SELECT impossible

171

Les triggers

Trigger INSTEAD OF

- Permet de remplacer une modification sur une table par une action spécifique
- Les variables old et new peuvent être utilisées comme si l'événement avait lieu
- Utilisation possible pour mettre à jour une vue

172

Les triggers

Suppression

- **DROP TRIGGER** nom_trigger **ON** nom_table

Existe d'autres fonctions selon les SGBD comme

- ALTER
- ou REPLACE

173

Les triggers

Déclenchement peut provoquer la vérification des contraintes

Les triggers peuvent être déclenchés par des ON DELETE CASCADE

Cascade de triggers

Bouclage:

- Un trigger peut boucler... Arrêter la session pour réinitialiser...

174

Triggers System

Sur SCHEMA ou DATABASE

- Sur évènement LDD
- Sur évènement BD

BEFORE | AFTER | INSTEAD OF CREATE

175

Trigger SCHEMA

Evènement :

ALTER	DROP
ANALYZE	GRANT
ASSOCIATE STATISTICS	NOAUDIT
AUDIT	RENAME
COMMENT	REVOKE
CREATE	TRUNCATE
DISASSOCIATE STATISTICS	DDL

176

Exemple Oracle

```
CREATE OR REPLACE TRIGGER drop_trigger
  BEFORE DROP ON hr.SCHEMA
  BEGIN
    RAISE_APPLICATION_ERROR ( num => -20000,
                               msg => 'Cannot drop object' );
  END;
/
```

177

Trigger Schema

INSTEAD OF CREATE

- Événement déclencheur une instruction CREATE.

Exemple Oracle

```
CREATE OR REPLACE TRIGGER t
  INSTEAD OF CREATE ON SCHEMA
  BEGIN
    EXECUTE IMMEDIATE 'CREATE TABLE T (n NUMBER, m NUMBER)';
  END;
/
```

178

Trigger BD

Syntaxe :

CREATE TRIGGER nom_Trigger

[AFTER LOGON | BEFORE LOGOFF | AFTER SERVERERROR]

ON DATABASE

179

Exemples Oracle 1 et 2

```
CREATE OR REPLACE TRIGGER
check_user
  AFTER LOGON ON DATABASE
  BEGIN
    check_user;
  EXCEPTION
    WHEN OTHERS THEN
      RAISE_APPLICATION_ERROR
        (-20000, 'Unexpected
error: '||
DBMS_Utility.Format_Error_Stack);
  END; /
```

```
CREATE TRIGGER log_errors
  AFTER SERVERERROR ON DATABASE
  BEGIN
    IF (IS_SERVERERROR (1017))
    THEN
      NULL; -- (substitute code
that processes logon error)
    ELSE
      NULL; -- (substitute code
that logs error code)
    END IF;  END;  /
```

180

SYS_CONTEXT

Fonction permettant d'avoir des informations contextuelles sur l'environnement de la session en cours

Appel	Description
SYS_CONTEXT('USERENV', 'SESSION_USER')	Nom de l'utilisateur Oracle de la session
SYS_CONTEXT('USERENV', 'HOST')	Nom de la machine cliente
SYS_CONTEXT('USERENV', 'IP_ADDRESS')	Adresse IP du client
SYS_CONTEXT('USERENV', 'OS_USER')	Nom de l'utilisateur système (OS)
SYS_CONTEXT('USERENV', 'TERMINAL')	Nom du terminal
SYS_CONTEXT('USERENV', 'AUTHENTICATION_TYPE')	Type d'authentification (mot de passe, certificat, etc.)

181

Paramètre	Description	Paramètre	Description
SESSION_USER	Nom de l'utilisateur Oracle connecté	CURRENT_SQL	SQL en cours d'exécution (si activé)
CURRENT_USER	Utilisateur courant (peut différer avec AUTHID CURRENT_USER)	ENTRYID	ID d'entrée d'audit
OS_USER	Nom de l'utilisateur du système d'exploitation	PROXY_USER	Utilisateur proxy (si utilisé)
AUTHENTICATED_IDENTITY	Identité authentifiée (utile avec Enterprise User Security)	PROXY_USERID	ID de l'utilisateur proxy
AUTHENTICATION_TYPE	Type d'authentification (PASSWORD, EXTERNAL, etc.)	SESSION_USERID	ID de l'utilisateur Oracle
AUTHENTICATION_DATA	Données d'authentification (rarement utilisées directement)	CURRENT_USERID	ID de l'utilisateur courant
IP_ADDRESS	Adresse IP du client	DB_DOMAIN	Domaine de la base de données
HOST	Nom de la machine cliente	DB_NAME	Nom de la base de données
TERMINAL	Nom du terminal	INSTANCE	Nom de l'instance
NETWORK_PROTOCOL	Protocole réseau utilisé (TCP, etc.)	INSTANCE_NAME	Nom de l'instance Oracle
CLIENT_IDENTIFIER	Identifiant client défini via DBMS_SESSION.SET_IDENTIFIER	SERVER_HOST	Nom de l'hôte du serveur Oracle
CLIENT_INFO	Infos client définies via DBMS_APPLICATION_INFO.SET_CLIENT_INFO	SERVICE_NAME	Nom du service Oracle
MODULE	Nom du module (défini via DBMS_APPLICATION_INFO.SET_MODULE)	OS_PID	PID du processus client
ACTION	Action en cours (définie via DBMS_APPLICATION_INFO.SET_ACTION)	FG_JOB_ID	ID du job en premier plan
SESSIONID	ID de session Oracle	BG_JOB_ID	ID du job en arrière-plan
SID	Identifiant de session (identique à SESSIONID)	AUDITED_CURSORID	Curseur audité
LANGUAGE	Langue et territoire (ex: AMERICAN_AMERICA.WE8MSWIN1252)	AUDITED_LOGIN	Login audité
LANG	Langue (ex: AMERICAN)		
ISDBA	Retourne TRUE si l'utilisateur est DBA		
CURRENT_SCHEMA	Schéma courant		

182

Gestion des Jobs

183

Création d'un job

Fonction CREATE_JOB

- Nom et type du programme
- Date et type de répétition

Nécessite le privilège CREATE JOB

Exemple

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB (
    job_name      => 'JOB_NAME',
    job_type      => 'PLSQL_BLOCK',
    job_action    => 'BEGIN ...; END;',
    start_date    => SYSTIMESTAMP,
    repeat_interval => 'FREQUENCY=HOURLY; INTERVAL=1',
    enabled       => TRUE );
END; /
```

184

Création d'un job

CREATE_PROGRAM

```
BEGIN
  DBMS_SCHEDULER.CREATE_PROGRAM (
    program_name => 'PROG_NAME',
    program_type => 'PLSQL_BLOCK',
    program_action => 'BEGIN ...; END;' );
END;
```

Surcharge de CREATE_JOB

```
BEGIN
  DBMS_SCHEDULER.CREATE_JOB (
    job_name      => 'JOB_NAME',
    program_name  => 'PROG_NAME',
    start_date    => SYSTIMESTAMP,
    repeat_interval => 'FREQ=DAILY',
    enabled       => TRUE );
END;
```

185

Création d'un job avec arguments

CREATE_PROGRAM

```
DBMS_SCHEDULER.CREATE_PROGRAM (  
  program_name => 'PROG_NAME',  
  program_type => 'STORED PROCEDURE',  
  program_action => 'EMP_REPORT' );
```

Définition d'un argument

```
DBMS_SCHEDULER.DEFINE_PROGRAM_ARGUMENT (  
  program_name => 'PROG_NAME',  
  argument_position => 1,  
  argument_name => 'DEPT_ID',  
  argument_type => 'NUMBER', default_value => 50 );
```

Création d'un job spécifiant le nombre d'arguments

```
DBMS_SCHEDULER.CREATE_JOB( job_name => 'JOB_NAME',  
  program_name => 'PROG_NAME',  
  start_date => SYSTIMESTAMP, repeat_interval => 'FREQ=DAILY',  
  number_of_arguments => 1, enabled => TRUE);
```

186

Création d'un job avec arguments

CREATE_SCHEDULE

```
BEGIN  
  DBMS_SCHEDULER.CREATE_SCHEDULE (  
    schedule_name => 'SCHED_NAME',  
    start_date => SYSTIMESTAMP,  
    repeat_interval => 'FREQ=DAILY',  
    end_date => SYSTIMESTAMP + 15 );  
END;
```

CREATE_JOB

```
BEGIN  
  DBMS_SCHEDULER.CREATE_JOB (  
    job_name => 'JOB_NAME',  
    schedule_name => 'SCHED_NAME',  
    job_type => 'PLSQL_BLOCK',  
    job_action => 'BEGIN ...; END;',  
    enabled => TRUE );  
END;
```

187

Intervalle d'un job

Expression calendaire

```
repeat_interval => 'FREQ=HOURLY; INTERVAL=4'  
repeat_interval => 'FREQ=DAILY'  
repeat_interval => 'FREQ=MINUTELY; INTERVAL=15'  
repeat_interval => 'FREQ=YEARLY; BYMONTH=MAR, JUN, SEP, DEC; BYMONTHDAY=15'
```

Expression PL/SQL

```
repeat_interval => 'SYSDATE + 36/24'  
repeat_interval => 'SYSDATE + 1'  
repeat_interval => 'SYSDATE + 15/(24*60)'
```

188

Nommage

Du programme: CREATE_PROGRAM

De l'ordonnancement: CREATE_SCHEDULE

Exemple

```
BEGIN  
  DBMS_SCHEDULER.CREATE_JOB (  
    job_name => 'JOB_NAME',  
    program_name => 'PROG_NAME',  
    schedule_name => 'SCHEDULE_NAME',  
    enabled => TRUE  
  );  
END;  
/
```

189

Gestion des jobs

Lancement

```
DBMS_SCHEDULER.RUN_JOB('SCHEMA.JOB_NAME');
```

Arrêt

```
DBMS_SCHEDULER.STOP_JOB('SCHEMA.JOB_NAME');
```

Arrêt en cours d'exécution

```
DBMS_SCHEDULER.DROP_JOB('JOB_NAME', TRUE);
```

Vues du dico des données

```
[DBA | ALL | USER]_SCHEDULER_JOBS
```

```
[DBA | ALL | USER]_SCHEDULER_RUNNING_JOBS
```

```
[DBA | ALL | USER]_SCHEDULER_JOB_CLASSES
```

```
[DBA | ALL | USER]_SCHEDULER_JOB_LOG
```

```
[DBA | ALL | USER]_SCHEDULER_RUN_DETAILS
```

```
[DBA | ALL | USER]_SCHEDULER_PROGRAMS
```